State Space Models SSMs and Mamba

In transformers each token attend to all other tokens
At inference they became slow quadratic complexity
KV cache (memory) also grows with sequence length.

RNNs are fast at inference and requires less memory as its current state depends only on previous state.
But RNNs have catastraphic forgetting. older information gets pushed out
By t=20 information from early tokens is nearly gone.
Hence cant use for long sequences.

Transformers: ~~Perfect~~ Perfect memory but slow at inferrence
RNNs: Fast inferrence but limited memory
Can we combine both?
What is state space model?
State variables: Minimum set of variables needed to fully describe system.
State space: N-dimensional space containing all possible state values
State vector: A point in state space representing current state (condition)

e.g. a car on a track
State variable: [position, velocity]
State vector: [5.3m from start, 20m/s]

SSMs: Attempts to predict next state. They contain info of state representations.
State Equation:

$$h'(t) = A \cdot h(t) + B \cdot x(t)$$

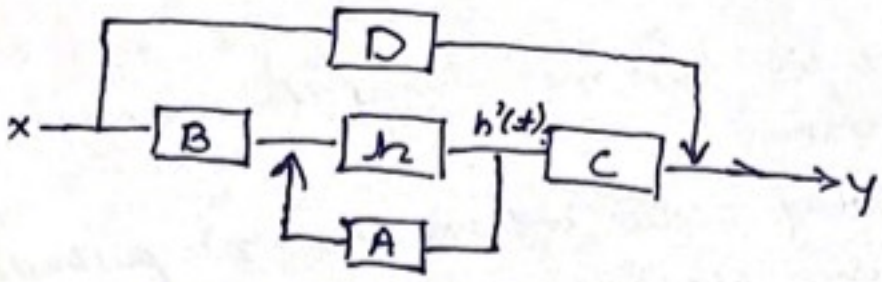
How state changes based on input
$x(t)$ = input signal at time t
B = How input affects the state.
$h(t)$ = hidden state latent vector.
A = state matrix. How state evolves over time.
$h'(t)$ = state derivative. Rate of change of state w.r.t. time.

<u>output equation</u> : How state is translated (mapped) to output.

$y(t) = C h(t) + D x(t)$

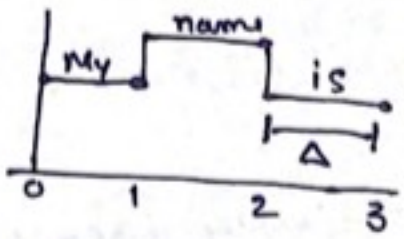
C = output matrix. How state is mapped to o/p.

D = skip connection. direct i/p to o/p. How i/p affects o/p.



At this point this state space model is only processing continuous ~~discrete~~ time signal. But our i/p is always discrete.

To convert it (SSM) to process discrete time signal, we use 0 order hold technique. we hold each discrete signal value untill next value arrives



This converts discrete signal to continuous signal.
A = step size. This is a learned parameter.

Using this we discretise Matrix A and B in state equation.

$\bar{A} = \exp(\Delta A)$

$\bar{B} = (\Delta A)^{-1}(\exp(\Delta A) - I) \cdot \Delta B$

we can again use step size to convert continuous signal back to discrete signal.

$h_K = \bar{A} h_{K-1} + \bar{B} x_K$ – State equation (discretised)

$y_K = C h_K$ – o/p equation (discritised)

At timestep 0.

$h_0 = \bar{B} x_0$

$y_0 = C h_0$

Time step 1

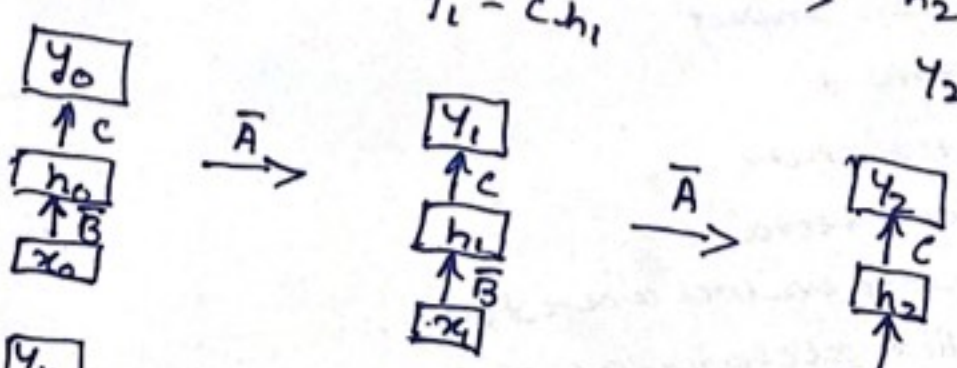
$\rightarrow h_1 = A h_0 + B x_1$

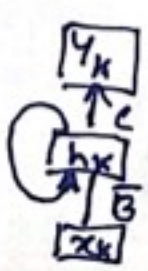
$y_1 = C h_1$

Time step 2

$\rightarrow h_2 = A h_1 + B x$

$y_2 = C h_2$.





RNN representation of SSM

These are hard to parallelise.

Hence we use convolution which is good at parallelising.

we use convolution mode during training.

we use recurrent mode during inference.

There is an issue with these representations.

All matrices A, B and C are fixed for all timesteps.

It is a static representation that is not content aware.

They cannot represent long range dependencies, they forget information from distant past.

We use <u>HiPPO: High order Polynomial projection order.</u>

Compress history to fixed size state vector. Using polynomial projections.

For state dimension N, the HiPPO A matrix has a specific structure.

$$A[n,k] = (2n+1)^{1/2} * (2k+1)^{1/2} * \int_0^1 P_n(x) * P_k(x)\,dx$$ where P_n are legendre polynomials

Simplified HiPPO (S_4)

$A_n = -(n+1)$ for $n = 0, 1, 2, \ldots, N-1$

This creates exponential decay rates that preserve information at different time scales.

Impact of HiPPO

From S_4 paper: simply modifying an SSM from a random matrix A to HiPPO matrix improved performance on sequential MNIST from 60% to 80%.